

Quand l'enveloppe suffit : falsification pré-enregistrée du durcissement par l'incertitude dans un moniteur d'assurance runtime pour satellites autonomes

Sami Bey

Ingénieur HES-SO — Haute École Spécialisée de Suisse occidentale

Epalinges, Vaud, Suisse — sami.bey@pedt-technologies.com

Implémentation de référence et données de campagne : github.com/Stab123/runtime-assurance-monitor (MIT)

Document de travail — septembre 2026

Version 3 — septembre 2026. Chiffres post-correction : deux défauts de code et un défaut de documentation trouvés après la v2 ; toutes les campagnes rejouées à configurations strictement inchangées — conclusions qualitatives inchangées (protocole complet dans le CHANGELOG du dépôt). Cette version ajoute la campagne P3.1 (section 5.4) et remplace la v2. Code, données et historique archivés : DOI 10.5281/zenodo.22552147.

Résumé

Nous présentons la conception, l'implémentation et la campagne de falsification pré-enregistrée d'un moniteur d'assurance runtime pour satellites autonomes. Le moniteur supervise une couche de décision non vérifiable : à chaque cycle de décision, il émet exactement un verdict — autoriser, modifier minimalement, replier, ou déclarer la situation indéterminée — et enregistre une trace de décision autodescriptive. Sa sémantique de modification spécialise le filtre de sécurité prédictif de Wabersich et Zeilinger au cas d'une action scalaire, avec une dichotomie à nombre d'itérations fixe et une politique de repli déterministe, vérifiable statiquement ; son scénario et sa cadence suivent l'expérience en vol de vérification runtime CySat-I. Le pari d'ingénierie testé (hypothèse H4) est que le durcissement par l'incertitude de premier ordre — évaluation pessimiste des trajectoires à $x \mp k_\sigma \sigma$ et repli lorsque l'incertitude estimée dépasse un seuil par contrainte — achète de la robustesse à un coût opérationnel acceptable. Plutôt que d'ajuster le mécanisme jusqu'à ce qu'il paraisse bon, nous avons figé un critère de falsification et une configuration versionnée, puis mené une campagne Monte Carlo (quatre bras, nombres aléatoires communs, estimation d'incertitude par innovations qui ne lit jamais les paramètres du modèle de prédiction, paramètres de plante tirés indépendamment et délibérément défavorables au moniteur). Le bras à enveloppe seule ne produit aucune violation de sécurité sur aucun run ; tout niveau d'activation de H4 coûte strictement plus de repli et strictement moins de livraison mission, sans bénéfice de sécurité. Un quatrième bras isole le coût du pessimisme d'évaluation de celui du seuil d'incertitude. Nous énonçons le résultat négatif avec sa portée exacte — y compris la raison structurelle pour laquelle l'enveloppe suffit ici. Une seconde campagne fait varier le seul levier structurel identifié — le délai d'armement du repli, mesuré par le ratio $r = \tau_{\text{armement}} / \tau_{\text{violation}}$ ($\tau_{\text{violation}}$ formel : 205,8 s sur le modèle nominal) — de 0,58, sa valeur en P2, jusqu'à 0,92 : l'enveloppe seule reste à zéro violation sur toute la bande compilable, et au-delà de $r \approx 0,95$ le jeu de contraintes est refusé à la compilation — le régime où le durcissement pourrait se justifier est non déployable par construction. Nous rapportons enfin les sept défauts trouvés alors que la suite de tests unitaires était entièrement verte — quatre défauts d'interaction avant la première campagne, puis deux défauts de code et un défaut de documentation — et le protocole de rejoué à configurations inchangées qui a suivi les trois derniers.

Mots-clés : assurance runtime — filtre de sécurité prédictif — vérification runtime — satellites autonomes — falsification — campagne Monte Carlo — nombres aléatoires communs — traçabilité de décision

1. Introduction

Les satellites autonomes embarquent de plus en plus de couches de décision — planificateurs, contrôleurs appris, optimiseurs de ressources — dont le comportement n'est pas certifiable par les moyens classiques. La réponse de l'assurance runtime (RTA) à ce problème est architecturale : laisser la couche complexe proposer, et placer sur le chemin de ses actions un composant simple et vérifiable, capable d'autoriser, de modifier ou de bloquer [2][3]. Le présent travail s'inscrit dans le cadre d'un moniteur d'assurance embarqué spécifié pour l'autonomie décisionnelle sous contrainte de calcul spatial (spécification interne RAM-SPEC-0001 [8]), motivé par l'axe « Runtime-Assured Autonomous Satellite » identifié dans le contexte du programme S4 (ESA Phi-Lab Suisse / ESDI) [11].

Deux lignées de travaux fondent la conception. Du filtre de sécurité prédictif de Wabersich et Zeilinger [4][5], le moniteur retient la sémantique de la *modification minimale* : une action candidate qui mènerait à une violation de contrainte est remplacée par l'action la plus proche qui conserve une trajectoire de backup sûre, et seule l'infaisabilité de ce problème déclenche le repli. De l'expérience CySat-I d'Aurandt, Jones et Rozier [1] — première démonstration en vol d'une vérification runtime déclenchant une récupération de faute autonome sur un CubeSat — il retient la méthodologie : contraintes dérivées d'exigences en langage naturel, cadence de monitoring de 5 secondes, injection délibérée de fautes, récupération par stratégies prédéfinies.

Le moniteur construit durcit l'évaluation prédictive par l'incertitude de premier ordre : les trajectoires prédites sont évaluées à la borne défavorable $x \mp k_\sigma \sigma$ de l'état estimé, et une incertitude estimée excessive force le repli quelle que soit la valeur nominale (hypothèse H4). H4 est un pari d'ingénierie : il devrait acheter de la robustesse à l'écart modèle-monde à un coût acceptable en taux de repli et en livraison mission. Ce genre de pari est d'ordinaire « validé » en exhibant des scénarios où il aide. Nous avons choisi la discipline opposée, plus proche de Popper que de l'ajustement : figer un critère de falsification et une configuration versionnée *avant* la campagne, tirer les paramètres de plante dans des bornes délibérément défavorables au moniteur, et s'engager à conserver le résultat quel qu'il soit.

Le résultat est négatif, et net. Un moniteur à enveloppe seule (sans durcissement) ne produit aucune violation de sécurité sur aucun run de la campagne ; tout niveau d'activation de H4 coûte strictement plus de repli et strictement moins de livraison mission, sans aucun bénéfice de sécurité en regard. Comme le critère de falsification était figé à l'avance et que

le bras de contrôle est à son plafond — zéro violation est le meilleur résultat possible — aucun ajustement de seuil ne peut sauver l'hypothèse dans ce régime.

Contributions. (i) Une implémentation de référence de la sémantique du moniteur — quatre verdicts, spécialisation scalaire du filtre de sécurité prédictif, validation à la compilation de l'atteignabilité du repli, trace de décision autodescriptive — avec traçabilité au niveau exigence vers la spécification. (ii) Une méthodologie de falsification pour mécanismes d'assurance : critères pré-enregistrés, nombres aléatoires communs avec témoin par run, estimation d'incertitude par innovations sans circularité, tirages de plante indépendants et défavorables, artefacts bit-reproductibles (configurations versionnées, empreintes de code embarquées dans les résultats). (iii) Un résultat négatif correctement borné sur le durcissement par l'incertitude de premier ordre, incluant un quatrième bras expérimental qui sépare le coût du pessimisme d'évaluation de celui du seuil d'incertitude, un critère structurel délimitant le régime dans lequel le résultat vaut (section 7), et un rapport des sept défauts que les tests unitaires n'ont pas vus — quatre d'interaction avant la première campagne, trois de plus (deux de code, un de documentation) en septembre 2026, suivis d'un rejeu complet à configurations inchangées.

Ce que ce papier ne prétend pas. Il ne montre pas que le durcissement par l'incertitude est inutile en général. Il montre que *dans un régime où l'enveloppe de sécurité seule suffit déjà, le durcissement de premier ordre ne peut pas justifier son coût* — un énoncé plus faible et plus vrai, et celui que l'expérience était réellement conçue pour tester.

2. Contexte et travaux liés

2.1 Assurance runtime

L'architecture Simplex a introduit le schéma consistant à adjoindre à un contrôleur performant non vérifié un contrôleur de haute assurance vérifié, plus une logique de commutation qui rend la main au côté sûr lorsque la plante approche la frontière de la région récupérable [2] ; la formulation de Sha — « utiliser la simplicité pour contrôler la complexité » — sépare les exigences critiques, confiées à du logiciel simple, des propriétés désirables, déléguées au logiciel complexe [3]. Simplex *commute* vers le contrôleur de backup. Le moniteur étudié ici *filtre* : son intervention première est une action modifiée qui reste aussi proche que possible de la candidate, le repli étant réservé à l'infaisabilité — ce qui est précisément la sémantique du filtre de sécurité prédictif.

2.2 Filtre de sécurité prédictif

Wabersich et Zeilinger certifient des contrôleurs appris en résolvant, à chaque pas, un problème contraint à horizon fini

qui recherche une trajectoire de backup sûre vers un ensemble sûr, en ne modifiant l'entrée proposée qu'autant que nécessaire [5] ; le cas linéaire a paru dans [4]. La section 3.2 rappelle la formulation et énonce nos trois écarts délibérés : une politique de backup non optimisée mais vérifiable statiquement, une spécialisation scalaire qui remplace le programme quadratique par une dichotomie à nombre d'itérations fixe, et une évaluation pessimiste à un niveau de confiance déclaré (non calibré).

2.3 Vérification runtime en vol : CySat-I

Aurandt, Jones et Rozier ont fait voler le moteur R2U2 sur le calculateur du CubeSat CySat-I : 22 spécifications LTL à temps de mission élicitées à partir d'exigences en langage naturel sur le système de puissance (EPS), évaluées par une tâche FreeRTOS de 5 secondes, avec huit scénarios de fautes EPS émuloées déclenchant des récupérations autonomes — et un véritable bug de firmware découvert par le moniteur lors des tests au sol [1]. CySat-I vérifiait l'état du satellite contre des propriétés temporelles. Le moniteur présenté ici vérifie les actions d'une couche de décision autonome contre des contraintes de sécurité — un objet différent, la même discipline : des exigences en langage naturel compilées en forme vérifiable, une cadence fixe, des fautes injectées, une récupération prédéfinie. Notre scénario de démonstration (section 3.4) reprend délibérément leur cadre EPS.

2.4 Documents internes

Le travail est gouverné par trois documents de travail non publiés cités dans tout le texte : la spécification fonctionnelle RAM-SPEC-0001 [8], la note de calcul de budget de trace RAM-NOTE-0001 [9] (qui dimensionne la trace de décision à 16/48/160 octets par cycle et justifie la rétention étendue par événement au regard de la rareté des verdicts non nominaux), et une note de positionnement

ECSS RAM-ECSS-0001 [10], qui mappe les exigences du moniteur sur ECSS-E-ST-70-11C Rev.1 [6] et ECSS-E-ST-70-41C [7]. Un état de l'art sur l'assurance runtime embarquée [11] fournit le contexte programme. Aucun des résultats rapportés ci-dessous ne dépend du contenu de ces documents internes : l'implémentation de référence, les configurations figées et les résultats de campagne sont publics et auto-suffisants.

3. Le moniteur (P0)

3.1 Sémantique des verdicts et interfaces

Le moniteur est synchrone avec la boucle de décision (période de 5 s, selon la cadence de CySat-I). À chaque cycle, il émet *exactement un* verdict (exigence RA-FUN-001) : AUTORISÉ — l'action candidate est transmise inchangée ; MODIFIÉ — l'action admissible la plus proche est transmise à sa place ; REPLI — la politique de repli déterministe prend la main ; INDÉTERMINÉ — les entrées sont invalides ou périmées ; l'exécution le traite comme un repli, mais la trace le garde distinct (RA-FUN-002). La seule entrée de la couche de décision vers le moniteur est l'action candidate ; la trace n'en conserve qu'une empreinte de 3 octets, jamais d'état interne (RA-IND-001). Une candidate absente à l'échéance est elle-même un déclencheur de verdict : repli, cause TIMEOUT_ACTION (RA-IND-003). Le repli est verrouillé : le retour au nominal exige un critère explicite, compté et horodaté — N cycles consécutifs pleinement sûrs plus une durée minimale en repli (RA-FUN-006). Chaque verdict est notifié au FDIR plateforme (interface IF-4), et le jeu de contraintes ne peut être mis à jour que sur autorisation explicite, son numéro de version figurant dans chaque enregistrement ultérieur (IF-6, RA-IND-004).

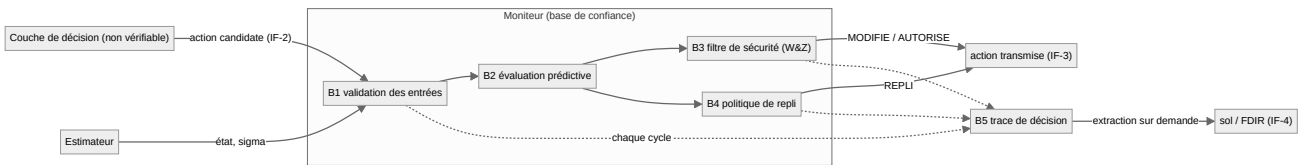


Figure 1. Architecture du moniteur (blocs B1–B5 de RAM-SPEC-0001). La couche de décision est une boîte noire ; seules ses actions candidates franchissent l'interface. L'enregistreur de trace n'est jamais sur le chemin critique : sa saturation dégrade la trace, jamais le verdict.

3.2 Spécialisation scalaire du filtre de sécurité prédictif

Le filtre de Wabersich et Zeilinger résout, à chaque instant de décision, le problème

$$\begin{aligned}
 & \min_{v_0, \dots, v_{N-1}} \|v_0 - u_{\text{cand}}\|^2 \\
 \text{s.c.} \quad & z_{k+1} = f(z_k, v_k), \quad z_0 = x(t) \\
 & z_k \in \mathcal{X}, \quad v_k \in \mathcal{U}, \quad z_N \in \mathcal{X}_f
 \end{aligned} \tag{1}$$

et applique v_0 ; l'infaisabilité donne la main à une trajectoire de backup. Notre spécialisation conserve la sémantique et remplace la machinerie, en trois étapes documentées. **(a)**

Backup = politique de repli. La trajectoire de backup n'est pas optimisée : c'est le déroulé de la politique de repli déterministe (extinction du payload), vérifiable statiquement — plus conservateur que le backup optimisé, délibérément, parce que cette politique fait partie de la base de confiance.

(b) Action scalaire. Pour une action scalaire sous hypothèse de monotonie (plus de courant payload n'est jamais plus sûr), l'ensemble admissible de v_0 est un intervalle $[u_{\min}, \hat{u}]$; la borne supérieure est obtenue par dichotomie en un nombre fixe d'itérations (32), ce qui borne statiquement le profil d'exécution dans l'esprit des exigences de ressources de la spécification. Le verdict MODIFIÉ transmet $\hat{u}$ — la modification minimale de la candidate au sens de (1).

(c) Évaluation pessimiste. Chaque contrainte est vérifiée à la borne défavorable $x \mp k_\sigma \sigma$ de l'état estimé, avec $k_\sigma = 3,0$ correspondant à un niveau de confiance déclaré $p_S \approx 99,7\%$ sous hypothèse gaussienne — déclaré, pas calibré : relier ce niveau aux propriétés réelles de l'estimateur est un travail futur, et d'ici là toute marge se lit « à p_S déclaré ».

3.3 Validation à la compilation

Deux exigences sont déchargées à la compilation du jeu de contraintes, pas en vol. L'horizon de prédiction couvre le délai d'armement maximal du repli plus une marge (RA-FUN-004) — un repli qui prend 120 s à s'armer n'est crédité qu'après ces 120 s dans la trajectoire évaluée, pendant lesquelles l'action transmise persiste. Et le repli doit être effectivement atteignable : depuis la frontière de garde de chaque contrainte, la trajectoire avec l'action admissible la plus défavorable persistée pendant l'armement (les deux bornes sont testées) ne doit franchir aucun seuil de sécurité

brut (RA-FUN-005). Un jeu invalide est refusé à la compilation par JeuInvalide. La marge de garde est le recul de commutation qui laisse au repli le temps d'agir — ce n'est pas la limite de sécurité, et le contrôle de compilation est délibérément mené contre les seuils bruts. Concrètement, sur le modèle de démonstration : un armement de 120 s passe (état de charge au pire en fin d'armement : 0,3583 pour un seuil de 0,35 — valeur recalculée après la correction de septembre 2026, la persistance appliquait l'action un pas de trop), un armement de 300 s est rejeté — le mur exact est à 196 s (section 5.4) — et la même contrainte aurait été acceptée si le contrôle avait ignoré le délai d'armement.

3.4 Scénario de démonstration et trace de décision

La démonstration reprend le cadre EPS de CySat-I : des exigences en langage naturel (« la charge batterie ne doit jamais passer sous le seuil de sous-tension », « la température batterie ne doit pas dépasser 45 °C ») compilées en deux contraintes scalaires avec marges de garde, délais d'armement et seuils d'incertitude par contrainte ; un scénario de deux orbites et demie avec une demande payload agressive et délibérément infaisable, et trois fautes injectées — une dérive d'action hors bornes, un estimateur dégradé, une couche de décision gelée. La figure 2 montre le comportement du moniteur : la candidate hors bornes est ramenée à la borne admissible ; la dégradation de l'estimateur déclenche le repli sur le seul motif d'incertitude ; le gel de la couche de décision déclenche le repli sur timeout ; et l'état de charge batterie longe le seuil de garde sans jamais franchir la limite de sécurité.

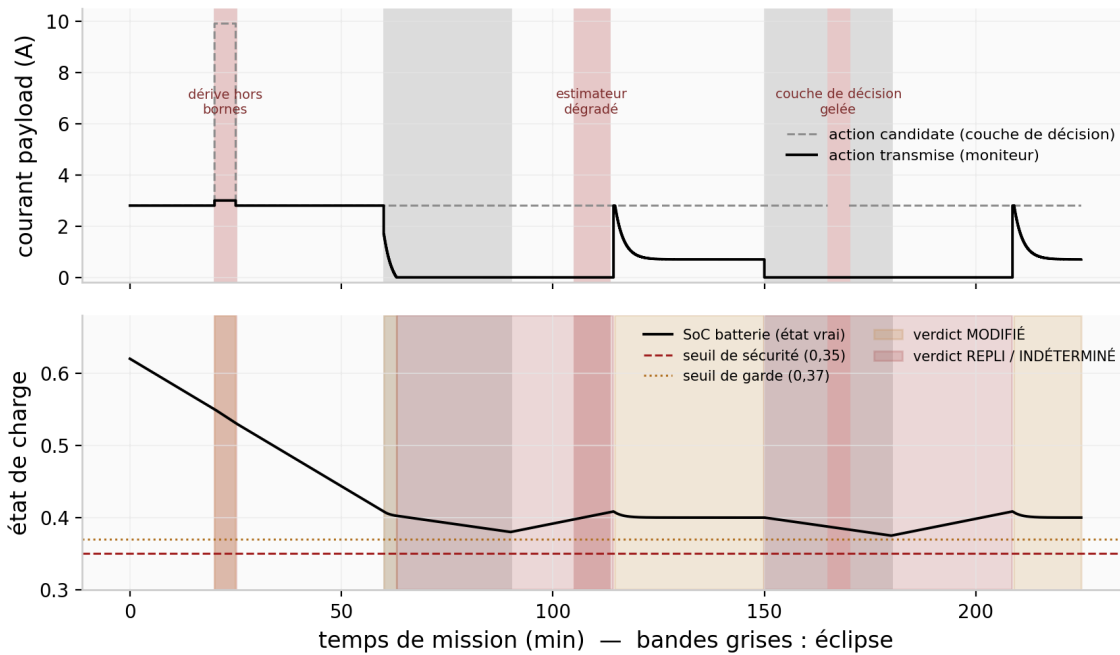


Figure 2. Scénario de démonstration (2,5 orbites, cycles de 5 s). Haut : action candidate proposée par la couche de décision (pointillés) et action effectivement transmise par le moniteur (trait plein) ; les bandes rouges marquent les trois fautes injectées, les bandes grises les éclipses. Bas : état de charge batterie vrai face au seuil de sécurité (0,35) et au seuil de garde (0,37) ; les zones ocre et rouge marquent les verdicts MODIFIÉ et REPLI/INDÉTERMINÉ. Le profil de demande est délibérément infaisable — le scénario est un test de stress, pas un plan d'opérations ; l'acceptabilité opérationnelle est l'objet de la section 5.

Un trait de cette figure vaut la peine d'être signalé avant la campagne, parce que c'est le point que la campagne renverse. Le repli sur incertitude visible pendant la dégradation de l'estimateur *ressemble* au mécanisme qui justifie sa place. La section 5 montre que dans ce régime il est redondant : l'enveloppe seule attrape les mêmes cas sans lui. La démonstration établit que le mécanisme se déclenche comme spécifié ; elle n'établit pas que son déclenchement prévient quoi que ce soit.

Chaque cycle est enregistré (RA-TRC-001) dans un enregistrement autodescriptif à format fixe — trois variantes de 16, 48 et 160 octets, dimensionnées par la note de budget de trace [9] — dans un tampon anneau à capacité fixe. Les enregistrements portent la version du jeu de contraintes, le verdict, une cause qui distingue le repli d'enveloppe du repli d'incertitude (RA-TRC-004), la contrainte déterminante et sa marge normalisée, les empreintes des actions candidate et transmise, et un CRC ; un enregistrement se decode de façon

autonome pour l'audit à l'aveugle, sans accès aux internes de la couche de décision (RA-TRC-002, RA-TRC-005). Un verdict non nominal fige une fenêtre de contexte en mission, vidant de son sens l'économie de rareté sur laquelle repose le budget de trace [9]. La saturation dégrade la trace, jamais le verdict (RA-RES-004), et chaque enregistrement perdu est compté (RA-RES-005). Un signal de vie par cycle est surveillé par un chien de garde indépendant (RA-SUR-002).

3.5 État de la vérification

L'implémentation de référence (Python, ~1 100 lignes avec les tests) est livrée avec 23 tests unitaires, chacun rattaché à une exigence. La table 1 résume la traçabilité. P0 n'est pas du code de vol : les bornes de mémoire et de temps d'exécution au pire cas sont seulement préparées (dichotomie à itérations fixes, tampons à capacité fixe), le modèle de prédiction est parfait, et l'action est scalaire ; ces dettes sont énoncées dans le README de l'implémentation et en section 7.

Table 1. Traçabilité exigence → test (condensée ; les identifiants renvoient à RAM-SPEC-0001 [8]).

Exigence	Mécanisme	Test
RA-FUN-001/002	exactement un verdict par cycle ; INDÉTERMINÉ tracé distinctement, exécuté comme un repli	test_ra_fun_001/002
RA-FUN-003	le seuil d'incertitude par contrainte force le repli quelle que soit la valeur nominale	test_ra_fun_003
RA-FUN-004/005	l'horizon couvre le délai d'armement ; atteignabilité du repli vérifiée à la compilation contre les seuils bruts, action au pire persistée pendant l'armement	test_ra_fun_004, test_ra_fun_005 (×2)
RA-FUN-006	repli verrouillé ; critère de retour explicite, compté, horodaté	test_ra_fun_006
RA-IND-001/003/004	action candidate seule ; empreinte dans la trace ; candidate absente → repli ; mise à jour du jeu autorisée et versionnée	test_ra_ind_*
RA-RES-004/005	la saturation de la trace ne bloque jamais un verdict ; pertes comptées	test_ra_res_004_005
RA-TRC-001..007	un enregistrement par cycle ; autodescriptif ; causes distinguées ; schéma versionné ; fenêtre figée sur repli seul ; rebouclage du tampon sans perte	test_paquet_c_*, test_tampon_reboucle_sans_pe rte, ...
RA-SUR-002	signal de vie par cycle + chien de garde indépendant	test_ra_sur_002

4. Méthodologie de falsification

4.1 L'hypothèse et le critère pré-enregistré

L'hypothèse H4 soutient que le durcissement par l'incertitude de premier ordre — évaluation pessimiste à $x \mp k_\sigma \sigma$ plus repli sur incertitude excessive — achète de la robustesse à un coût opérationnel acceptable.

H4 combine deux mécanismes distincts, et la campagne est conçue pour les départager. Le premier est l'évaluation pessimiste à $k_\sigma = 3$ (section 3.2c) ; le second est le seuil d'incertitude par contrainte, qui est l'exigence RA-FUN-003. Le bras D isole le coût du premier, et l'écart de D à C isole le coût du second. Falsifier H4 falsifie donc la conjonction, et la décomposition en bras attribue le coût à chaque composante séparément — une distinction que le README de l'implémentation de référence suit également.

Critère de falsification (figé, config P2.1).

H4 n'est confirmée que s'il existe un point de grille de seuil d'incertitude dont le taux de violation de sécurité est *strictement inférieur* à celui du bras à enveloppe seule, à coûts de repli et de livraison comparables. Si le bras à enveloppe seule est à zéro violation, aucun point de grille ne peut satisfaire le critère, et H4 est falsifiée dans ce régime.

Trois métriques opérationnelles sont enregistrées par run : le **taux de violation** (fraction de cycles où l'état *vrai* de la plante franchit un seuil de sécurité *brut*), le **taux de repli** (fraction de cycles de verdict REPLI ou INDÉTERMINÉ), et la **livraison mission** (énergie effectivement livrée au payload sur énergie demandée pendant les fenêtres payload). Les intervalles de confiance sont des intervalles de Student à 95 % sur les taux par run ; les proportions de runs utilisent des intervalles de Wilson.

4.2 Bras et nombres aléatoires communs

Quatre bras partagent chaque tirage. **A** : sans moniteur — la candidate (bornée) commande directement la plante. **B** : enveloppe seule — le moniteur avec $k_\sigma = 0$ et un seuil d'incertitude infini. **C** : H4 complet — $k_\sigma = 3$ et le seuil d'incertitude fixé à un point de grille. **D** (ajouté en deuxième version de campagne, cf. 5.2) : pessimisme d'évaluation seul — $k_\sigma = 3$, seuil infini. La comparabilité repose sur les nombres aléatoires communs : pour le run i , les paramètres de plante sont tirés une fois depuis `graine_plante + i`, et le bruit de mesure est régénéré à l'identique dans chaque bras depuis `graine_bruit + i`, avec exactement deux tirages gaussiens par cycle dans tous les bras (l'estimateur tourne aussi dans le bras A, inutilisé). Les deux graines de base sont séparées de plus que le nombre de tirages qu'aucun run ne consomme, de sorte que les flux de plante et de bruit ne peuvent pas se chevaucher. Seule la configuration du moniteur change. Un témoin par run — le tirage de bruit brut, stocké avant toute arithmétique — est comparé entre bras ; il a concordé sur 32/32 runs des deux premières campagnes et 300/300 de la troisième. Deux conceptions antérieures du témoin ont échoué honnêtement et méritent d'être rapportées : prendre l'état *estimé* comme témoin échoue parce que les estimations divergent entre bras par construction (cette divergence est l'objet de l'expérience), et reconstruire le tirage par soustraction en virgule flottante n'est pas bit-identique entre bras.

4.3 Estimation d'incertitude sans circularité

Le point critique de la conception est l'origine de σ . Si σ dérivait de la même information que celle que le moniteur utilise pour prédire, le raisonnement serait circulaire et la campagne ne mesurerait rien. L'estimateur calcule donc σ à partir des *résidus de mesure uniquement*. Avec z la mesure, x^- la prédiction à un pas sous le modèle nominal, et

l'innovation $\nu = z - x^-$, l'estimateur applique une correction proportionnelle et maintient des statistiques exponentielles de l'innovation :

$$\sigma_i = \sqrt{\text{EWMA}_\beta(\nu_i^2)} + |\text{EWMA}_\beta(\nu_i)| \times H(2)$$

où H est l'horizon de prédiction du moniteur en pas. Le premier terme couvre le bruit de mesure et l'erreur instantanée ; le second propage sur l'horizon le biais systématique estimé — par exemple une dérive lente modèle-monde comme une capacité batterie réelle inférieure à la nominale. Les deux termes portent les unités de l'état mesuré, et la valeur absolue garde σ non négatif sous un biais signé. La seule information partagée avec le moniteur est le modèle nominal de propagation, ce qui est inévitable : les innovations sont calculées contre le modèle même avec lequel le moniteur prédit, donc σ mesure exactement l'erreur de prédiction du moniteur. σ ne lit jamais les paramètres de la plante — l'écart modèle-monde ne l'atteint que par les mesures. À la cadence de 5 s, la dérive paramétrique par cycle se situe sous le bruit de mesure unitaire ; elle ne devient visible que par le biais moyenné, raison d'être du second terme.

4.4 Paramétrisation de la plante — délibérément défavorable

Pour chaque run, les paramètres de la plante vraie sont tirés uniformément (le choix le moins informatif — aucune queue inventée) dans les bornes de la table 2, tandis que le moniteur conserve le modèle nominal P0. Le tirage est délibérément défavorable : le moniteur suppose une batterie de 10 Ah et un bus de 0,5 A, la plante tire 5–9 Ah et 0,45–0,6 A — le moniteur est systématiquement optimiste sur la capacité, et chaque run mesure exactement l'écart modèle-monde que H4 était censé absorber. Une fenêtre de dégradation multiplie le bruit de mesure par six pendant 45 minutes de chaque run de 3,75 heures.

Table 2. Tirages des paramètres de plante (uniformes) face au modèle nominal du moniteur.

Paramètre	Basse	Haute	Nominal
Capacité batterie (Ah)	5,0	9,0	10,0
Courant de charge au soleil (A)	1,0	1,4	1,2
Consommation bus (A)	0,45	0,60	0,50
Coefficient d'échauffement (°C/(A·s))	$1,2 \times 10^{-3}$	$2,2 \times 10^{-3}$	$1,5 \times 10^{-3}$
Relaxation thermique (1/s)	3×10^{-4}	7×10^{-4}	5×10^{-4}
État de charge initial	0,50	0,65	—
Température initiale (°C)	15	25	—

4.5 Pourquoi les seuils opérationnels ne sont pas arbitrés

La configuration versionnée porte trois valeurs d'acceptation opérationnelle (un plafond de taux de violation, un plafond de taux de repli, un plancher de livraison mission) explicitement marquées *non arbitrées*. C'est une position délibérée, et nous l'énonçons parce qu'elle est inattaquable : tout chiffre écrit aujourd'hui, après avoir vu les résultats, serait un chiffre choisi en connaissance de cause — un relecteur le verrait, et il aurait raison de le voir. La conclusion de falsification ne dépend pas de ces valeurs : elle repose sur une comparaison entre bras — le bras B atteint zéro violation, rien ne peut faire mieux, et chaque point de grille du bras C coûte plus que B sur les deux autres métriques. Les seuils ne serviraient qu'à une autre question — « ce système est-il opérationnellement acceptable ? » — à laquelle ce travail ne peut pas répondre seul, car elle exige un opérateur satellite. Nous y voyons le bon crochet pour une discussion avec un laboratoire ou un industriel : voici les courbes ; où placeriez-vous le plancher ? La table 4 en donne d'ailleurs une illustration concrète : la livraison du bras B s'y établit à 90,81 % avec une borne basse d'intervalle de confiance à 89,95 % — un plancher fixé à 90 % serait raté de 0,05 point à la lecture prudente, et passé à la lecture centrale. Le choix du point de lecture fait déjà partie de la décision d'acceptation ; il n'appartient pas à ce travail.

4.6 Reproductibilité et pré-enregistrement

Chaque version de campagne est une configuration JSON figée (graines, bornes de plante, grille, seuils) dont l'historique de modification est en ajout seul : P2.1 (falsification), P2.2 (quatrième bras et grille étendue), P2.3 (renforcement statistique). Les fichiers de résultats embarquent les empreintes SHA-256 du code de campagne et du moniteur. Comme chaque bras re-graine son propre bruit, les versions ultérieures reproduisent les antérieures bit à bit sur leurs runs communs — vérifié explicitement : l'exécuteur parallèle (P2.3) reproduit les résultats sériels de P2.2 bit à bit, et les runs 0–31 de P2.2 sont reproduits dans P2.3. Le dépôt public sépare preuve et jeu : le répertoire empreintes/ conserve les octets exacts ayant produit les résultats (vérification uniquement), ram_p0/ et ram_p2/ en fournissent la copie exécutable à chemins relatifs, et le script `verifier empreintes.py` affiche pour chaque campagne les empreintes attendues et calculées, module par module ; un workflow d'intégration continue rejoue la suite de tests et cette vérification à chaque poussée.

Les empreintes établissent que les résultats rapportés ont été produits par la configuration et le code livrés avec elles ; elles n'établissent pas, à elles seules, que le critère de falsification précède la campagne, car un historique de dépôt peut être réécrit par son auteur. Cet horizon de confiance est

désormais couvert par deux archives indépendantes et non réinscriptibles : Zenodo, qui horodate chaque version publiée du dépôt (DOI de concept 10.5281/zenodo.22552147, un DOI par version), et Software Heritage, qui conserve le graphe git complet — l'ordre configuration-avant-résultats y est lisible pour P2.1, P2.2, P2.3 et P3.1. Les étiquettes de version sont annotées mais non signées GPG : l'horodatage et l'intégrité reposent sur ces deux archives. Le lecteur qui voudra vérifier l'affirmation de pré-enregistrement s'appuiera sur ces dépôts plutôt que sur la parole de l'auteur.

5. Résultats

5.1 Campagne P2.1 — la falsification

La première campagne (32 runs par point de grille, 2,5 orbites et 2 700 cycles par run) est rapportée dans la table 3 exactement telle qu'elle a tourné. Le bras non monitoré A viole la sécurité dans 8 runs sur 32 (5,36 % des cycles en moyenne). Le bras à enveloppe seule B ne viole rien — zéro violation sur chaque run. Comme le critère pré-enregistré exigeait un point de grille *strictement* sous B, et que B est à zéro, aucun point de grille ne peut se qualifier : H4 est falsifiée. Et le côté coût rend la falsification opposable plutôt que technique : B paie son zéro à 7,03 % de repli et 88,1 % de livraison, tandis que le meilleur point de grille de C paie 10,25 % de repli et 83,5 % de livraison — plus de coût, aucun bénéfice, sur les trois métriques à la fois.

Table 3. Campagne P2.1 (32 runs par point, 2 700 cycles par run) : la falsification. Le bras B — enveloppe seule — ne viole rien sur aucun run ; chaque point de grille du bras C (H4 complet) coûte plus de repli et moins de livraison. Les taux sont des moyennes par run avec intervalles de Student à 95 % ; les proportions de runs utilisent des intervalles de Wilson.

Bras	seuil σ	N	taux de viol. % [IC95]	runs avec viol. [Wilson]	taux de repli % [IC95]	livraison % [IC95]
A	—	32	5.36 [1.18, 9.54]	8/32 [0.133, 0.421]	0.00 [0.00, 0.00]	100.00 [100.00, 100.00]
B	—	32	0.00 [0.00, 0.00]	0/32 [0.000, 0.107]	7.03 [4.84, 9.23]	88.11 [84.26, 91.95]
C	0.010	32	0.00 [0.00, 0.00]	0/32 [0.000, 0.107]	39.20 [33.75, 44.66]	49.85 [42.44, 57.26]
C	0.020	32	0.00 [0.00, 0.00]	0/32 [0.000, 0.107]	23.22 [21.47, 24.97]	72.21 [68.97, 75.44]
C	0.030	32	0.00 [0.00, 0.00]	0/32 [0.000, 0.107]	18.69 [16.83, 20.55]	79.50 [75.81, 83.19]
C	0.050	32	0.00 [0.00, 0.00]	0/32 [0.000, 0.107]	10.36 [7.71, 13.01]	83.52 [78.92, 88.12]
C	0.080	32	0.00 [0.00, 0.00]	0/32 [0.000, 0.107]	10.25 [7.58, 12.92]	83.52 [78.92, 88.12]

Le taux de violation du bras A tombe de 5,36 % ici à 2,53 % à $N = 300$ (table 4). Les deux valeurs sont cohérentes — l'intervalle P2.1 [1,18 ; 9,54] contient l'estimation ultérieure — mais le lecteur qui compare les deux tables doit lire le chiffre à $N = 32$ comme celui qui est bruité, non comme la preuve d'un scénario modifié : graines, bornes et marges sont identiques.

5.2 Campagne P2.2 — corriger l'expérience, pas la conclusion

Deux défauts du plan expérimental P2.1 ont été identifiés en revue. D'abord, le bras C faisait varier deux choses à la fois : il activait le seuil d'incertitude et faisait passer k_σ de 0 à 3, de sorte que l'écart entre B et C confondait le coût du pessimisme d'évaluation avec celui du seuil. Ensuite, la grille saturait : le σ estimé dépassait rarement 0,05, donc le haut de la grille était déjà « H4 éteint » — les points 0,05 et 0,08 étaient bit-identiques (ils diffèrent légèrement depuis la correction de septembre 2026, sans conséquence : 9,60 contre 9,43 points de repli à $N = 300$, table 4). P2.2 a corrigé les deux, et seulement ceux-là : un quatrième bras D ($k_\sigma = 3$, seuil infini) isole le coût du pessimisme le long de la chaîne $B \rightarrow D \rightarrow C$, et la grille a été étendue vers le bas,

dans la région où σ vit réellement. Graines, bornes de plante, marges et nombre de runs n'ont pas été touchés ; P2.2 reproduit P2.1 bit à bit sur chaque point commun.

Deux constats en ont résulté. Mécaniquement, le bras D est identique à C@0,08 sur chaque run — preuve directe que le seuil ne s'y déclenche jamais, donc que tout l'écart B-vers-C@0,08 est du coût de pessimisme. Quantitativement, le pessimisme d'évaluation seul ($B \rightarrow D$) coûte, à $N = 32$, +3,3 points de repli et -4,6 points de livraison (l'estimation à $N = 300$ de la table 4 donne +2,9 et -5,0 points) ; le seuil ($D \rightarrow C$) ajoute jusqu'à +84 points de repli au point de grille le plus fin, où le système passe 94,6 % des cycles en repli et livre 4,7 % de l'énergie demandée. Aucune de ces corrections n'a sauvé H4 ; toutes deux en ont affiné la falsification. C'étaient des corrections du plan d'expérience — le scénario, les bornes et les marges n'ont jamais été réajustés, et les deux campagnes sont conservées intégralement.

5.3 Campagne P2.3 — renforcement statistique

Le critère de falsification était déjà tranché à $N = 32$, mais l'affirmation de zéro violation du bras B n'était bornée qu'à 10,7 % (borne supérieure de Wilson avec 0/32). P2.3 rejoue la configuration figée avec $N = 300$ — les runs 0–31

reproduisent P2.2 bit à bit — au moyen d'un exécuteur parallèle par run validé contre la campagne sérielle. Le bras B n'enregistre aucune violation en 300 runs (borne supérieure de Wilson à 95 % : 1,3 %, contre 10,7 % à $N = 32$), et le bras D ainsi que chaque point de grille du bras C non plus. La table 4 rapporte la grille complète — chiffres

du jeu post-correction (septembre 2026), qui diffèrent d'au plus 0,04 point des valeurs publiées en v2 ; la figure 3 compare les quatre bras sur les deux métriques de coût (le bras C est représenté au seuil $\sigma = 0,03$; la table 4 donne toute la grille).

Table 4. Campagne P2.3 (300 runs par point ; les runs 0–31 reproduisent P2.2 bit à bit). Le bras D isole le pessimisme d'évaluation ($k_\sigma = 3$, seuil infini) ; l'écart de D à une ligne C est le coût du seuil d'incertitude lui-même. Chiffres du jeu post-correction (septembre 2026) — ils diffèrent d'au plus 0,04 point des valeurs de la v2.

Bras	seuil σ	N	taux de viol. % [IC95]	runs avec viol. [Wilson]	taux de repli % [IC95]	livraison % [IC95]
A	—	300	2.53 [1.82, 3.24]	68/300 [0.183, 0.277]	0.00 [0.00, 0.00]	100.00 [100.00, 100.00]
B	—	300	0.00 [0.00, 0.00]	0/300 [0.000, 0.013]	6.57 [6.01, 7.14]	90.81 [89.95, 91.66]
D	∞	300	0.00 [0.00, 0.00]	0/300 [0.000, 0.013]	9.43 [8.73, 10.12]	85.83 [84.72, 86.93]
C	0.005	300	0.00 [0.00, 0.00]	0/300 [0.000, 0.013]	94.89 [94.38, 95.40]	4.54 [3.90, 5.19]
C	0.008	300	0.00 [0.00, 0.00]	0/300 [0.000, 0.013]	59.10 [56.99, 61.21]	31.91 [29.65, 34.17]
C	0.010	300	0.00 [0.00, 0.00]	0/300 [0.000, 0.013]	40.20 [38.53, 41.87]	48.00 [45.86, 50.14]
C	0.015	300	0.00 [0.00, 0.00]	0/300 [0.000, 0.013]	24.98 [24.45, 25.51]	67.34 [66.04, 68.64]
C	0.020	300	0.00 [0.00, 0.00]	0/300 [0.000, 0.013]	22.63 [22.14, 23.12]	74.13 [73.32, 74.95]
C	0.030	300	0.00 [0.00, 0.00]	0/300 [0.000, 0.013]	18.11 [17.59, 18.63]	81.52 [80.70, 82.33]
C	0.050	300	0.00 [0.00, 0.00]	0/300 [0.000, 0.013]	9.60 [8.91, 10.29]	85.82 [84.71, 86.93]
C	0.080	300	0.00 [0.00, 0.00]	0/300 [0.000, 0.013]	9.43 [8.73, 10.12]	85.83 [84.72, 86.93]

Campagne P2.3 — 300 runs par bras, nombres aléatoires communs (rejeu post-correction)

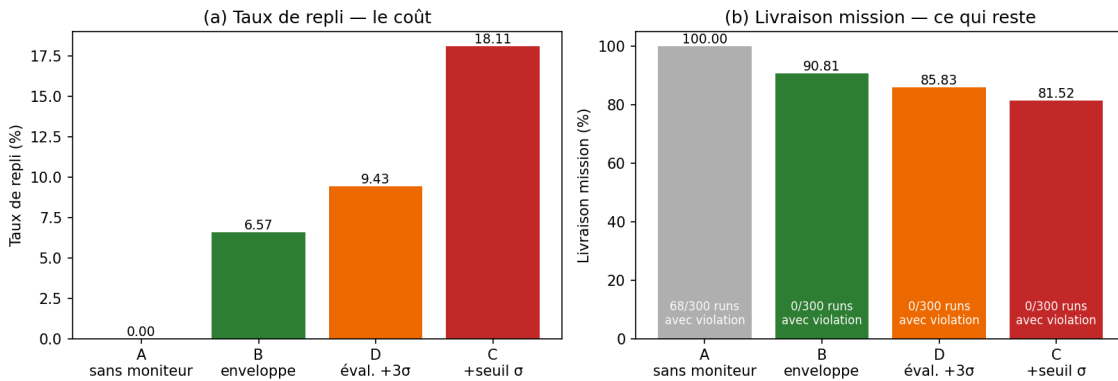


Figure 3. Campagne P2.3 (300 runs par bras, rejeu post-correction ; figure régénérée par figures/faire_figures.py depuis le fichier de campagne publié). Gauche : taux de repli par bras. Droite : livraison mission — l'annotation indique les runs avec violation (bras A : 68/300 ; B, D et C : 0/300, et zéro est le plafond). Le bras C est représenté au seuil $\sigma = 0,03$; la table 4 donne la grille complète.

5.4 Campagne P3.1 — pousser le ratio r vers le régime marginal

La section 7 conditionne l'utilité du durcissement à un régime où l'autorité du repli est marginale devant le temps-avant-violation, mesuré par le ratio $r = \tau_{\text{armement}} / \tau_{\text{violation}}$. Le $\tau_{\text{violation}}$ formel du scénario, mesuré sur le modèle nominal (depuis la frontière de garde, action la plus défavorable persistée, franchissement du seuil brut à 205,8 s), fixe l'échelle : en P2, $\tau_{\text{armement}} = 120$ s, soit $r = 0,58$ — le repli dispose de près du double du temps nécessaire. La campagne P3 pousse ce seul levier vers $r = 1$, tout le reste figé (graines, bornes de plante, marges, grille σ , cycles — configuration committée avant exécution).

Le régime marginal s'est révélé **non déployable par construction**. Le pilote de puissance (bras B seul, $N = 30$, exécuté avant figeage du critère) a d'abord localisé un mur : la validation à la compilation (RA-FUN-005, section 3.3) accepte le jeu de contraintes jusqu'à $\tau_{\text{armement}} = 195$ s inclus (marge résiduelle de +0,0002 sur l'état de charge) et le refuse dès 196 s — soit $r \approx 0,95$. P3.1 documente alors la bande compilable avec puissance statistique : trois points $\tau_{\text{armement}} \in \{140 ; 170 ; 190\}$ s, soit $r \in \{0,68 ; 0,83 ; 0,92\}$, $N = 300$, quatre bras, grille σ inchangée, avec un critère figé en deux clauses — la campagne n'est informative que si le bras B viole quelque part, et H4 n'est soutenue que si un couple (r , seuil σ) fait strictement mieux que B en violations à coûts comparables.

Résultat : le bras B reste à **zéro violation sur les trois points** (0/300 à chacun ; borne de Wilson 1,3 %), les bras C et D également, et le bras A — indépendant de r par construction — reproduit P2.3 bit à bit à chaque point (68/300 runs avec violation), comme la non-régression du protocole l'exigeait. La première clause du critère n'étant pas remplie, P3.1 est non concluante pour H4 — ce qui est exactement le point : là où le durcissement aurait sa chance, le moniteur ne compile pas. Le coût, lui, croît avec

τ_{armement} : le repli du bras B passe de 10,18 % à 15,60 % puis 18,77 %, et sa livraison de 89,05 % à 86,47 % puis 84,91 % (table 6, figure 4). Le mur est indexé sur le domaine d'action vérifié à la compilation ($u \in [0 ; 3]$ A) : l'énoncé vaut pour ce jeu de bornes, pas dans l'absolu. (La configuration figée indexait la grille sur une constante de référence $\tau_{\text{violation}} = 400$ s ; les r rapportés ici utilisent le $\tau_{\text{violation}}$ formel mesuré, 205,8 s.)

Table 6. Campagne P3.1 (300 runs par point, rejeu post-correction) : coûts en fonction du délai d'armement. B, D et C sont à zéro violation partout ; le bras A, indépendant de r par construction, reproduit P2.3 bit à bit à chaque point (2,53 % ; 68/300 runs) et n'est pas répété. Le bras C est représenté au seuil $\sigma = 0,03$.

τ_{armement} (s)	r	Bras	taux de repli % [IC95]	livraison % [IC95]
140	0,68	B	10.18 [9.40, 10.96]	89.05 [88.24, 89.86]
140	0,68	D	12.85 [12.00, 13.71]	83.94 [82.88, 84.99]
140	0,68	C	20.81 [20.10, 21.53]	79.53 [78.76, 80.29]
170	0,83	B	15.60 [14.64, 16.56]	86.47 [85.71, 87.23]
170	0,83	D	17.86 [16.85, 18.86]	81.46 [80.46, 82.47]
170	0,83	C	24.31 [23.38, 25.23]	77.04 [76.28, 77.79]
190	0,92	B	18.77 [17.71, 19.83]	84.91 [84.19, 85.63]
190	0,92	D	20.86 [19.77, 21.95]	79.86 [78.88, 80.85]
190	0,92	C	26.30 [25.26, 27.34]	75.40 [74.61, 76.18]

Campagne P3.1 — 300 runs par point, nombres aléatoires communs (rejeu post-correction)

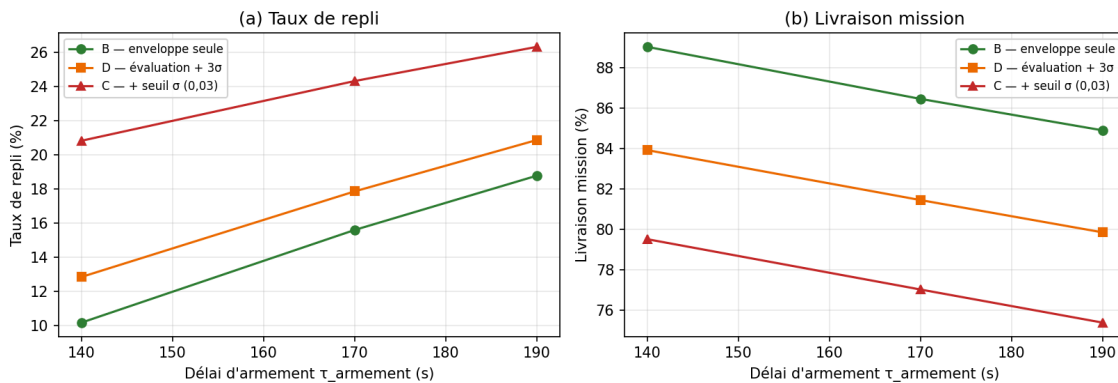


Figure 4. Campagne P3.1 (300 runs par point, rejeu post-correction ; figure régénérée par figures/faire_figures.py). Taux de repli (gauche) et livraison mission (droite) en fonction du délai d'armement du repli. Le coût de tous les bras monitorés croît avec τ_{armement} , et l'ordre $B < D < C$ est respecté à chaque point. Au-delà de 195 s ($r \approx 0,95$), le jeu de contraintes est refusé à la compilation.

6. Leçons apprises : ce que les tests unitaires n'ont pas vu

La suite de tests du moniteur — vingt tests à l'époque, vingt-trois depuis les tests de régression ajoutés en septembre 2026 — était entièrement verte lorsque la revue adverse a trouvé quatre défauts, tous dans les *interactions* entre composants plutôt qu'à l'intérieur d'un composant.

L'un des quatre mérite d'être énoncé hors du tableau, parce qu'il touche directement à la crédibilité du résultat. Le défaut 4 — l'appel interne de la dichotomie qui omettait silencieusement l'argument k_{σ} — aurait corrompu tout

bras à k_{σ} non standard, c'est-à-dire le bras B ($k_{\sigma} = 0$), le bras de contrôle sur lequel repose toute la falsification. S'il avait survécu jusqu'à la campagne, la base de zéro violation aurait été un artefact. Ce n'est pas le cas : les empreintes SHA-256 des modules du moniteur embarquées dans les fichiers de résultats P2.1, P2.2 et P2.3 sont identiques, et elles correspondent au code corrigé ; seul le pilote de campagne diffère entre versions. (Le pilote de P2.1 n'a pas survécu à l'environnement d'exécution — édité en place lors du passage à quatre bras ; son empreinte, conservée dans les résultats de P2.1, reste vérifiable, et la version quatre bras publiée dans empreintes/ reproduit ses bras A, B et C run

par run.) Aucun chiffre rapporté n'a été produit par ces quatre défauts-là, corrigés avant la première campagne — et les empreintes, recalculables par `verifier_empreintes.py` sur les octets d'empreintes/

permettent à un tiers de vérifier cette affirmation sans faire confiance à l'auteur. Cette phrase ne couvre en revanche pas les trois défauts trouvés plus tard (table 5, lignes 5–7) : voir le paragraphe de rejeu ci-dessous.

Table 5. Les sept défauts trouvés alors que la suite de tests était entièrement verte. Défauts 1–4 (interaction) : revue adverse, corrigés avant la première campagne — aucun chiffre publié n'en provient. Défauts 5–7 (septembre 2026, après la v2) : deux de code, un de documentation — les chiffres de cette version sont ceux du rejeu post-correction.

#	Défaut	Pourquoi les tests ne l'ont pas vu
1	Tampon anneau : sur une case figée, la tête d'écriture n'avancait pas — tout enregistrement postérieur à un gel était silencieusement perdu jusqu'à extraction.	Aucun test ne faisait reboucler le tampon ; le chemin de gel et le chemin de rebouclage étaient chacun testés isolément.
2	Gel de contexte déclenché sur tout verdict non nominal, y compris MODIFIÉ — la fenêtre figée fusionnait jusqu'à couvrir toute la mission (2 032 appels de gel), contredisant l'hypothèse de rareté du budget de trace.	Le mécanisme de gel était testé, son <i>économie</i> jamais — le défaut n'est visible que sur un scénario complet.
3	La validation du repli à la compilation passait un délai d'armement nul au contrôle de trajectoire — un armement de 300 s aurait été accepté alors que la contrainte est violée pendant l'armement.	Le contrôle existait et était testé — avec des paramètres qui le rendaient vacu. Un test dédié (300 s d'armement doivent lever <code>JeuInvalide</code>) verrouille désormais le comportement.
4	L'appel interne de la dichotomie omettait l'argument <code>k_sigma</code> , restaurant silencieusement le défaut — tout bras à k_σ non standard (bras B, $k_\sigma = 0$) aurait été corrompu.	Invisible tant qu'aucune expérience ne variait le paramètre ; trouvé en relisant le code <i>pour la campagne</i> , pas par la suite.
5	Persistance du repli d'un pas trop longue pendant l'armement ($k \leq \text{pas_armement} : \tau + 5 \text{ s}$ au lieu de τ) — convention partagée entre compilation et vol, donc aucun trou de sécurité ni biais entre bras, mais une étiquette temporelle fausse d'un pas (RA-FUN-004).	Les tests d'horizon partageaient la même convention fausse ; trouvée en relecture croisée avec l'exigence. Un test de régression verrouille désormais le comportement.
6	Projection d'une candidate sous <code>u_min</code> : le filtre renvoyait la borne <i>haute</i> de l'intervalle admissible au lieu de <code>u_min</code> — modification non minimale. Jamais atteint en campagne (candidates $\in \{0 ; 3\} A$).	Aucun test ne couvrait ce côté de l'intervalle, et le domaine des candidates de la campagne n'y passe jamais ; tests ajoutés des deux côtés.
7	Table σ du README figée sur des valeurs de brouillon, incohérentes avec le fichier de campagne — les données publiées n'ont jamais été affectées.	Aucun test ne relit la documentation ; trouvée pendant le rejeu, en comparant la table au fichier.

Le rejeu de septembre 2026. Les défauts 5 et 6 ont été trouvés après publication de la v2 — et les résultats publiés avaient, cette fois, été produits par le code affecté. Toutes les campagnes (P2.2, P2.3, pilote P3, P3.1) ont donc été rejouées sur l'intégration continue à configurations **strictement inchangées**, seul le code étant corrigé ; P2.1 n'a pas été rejouée (pilote historique perdu, plan déjà remplacé par P2.2) et ses octets d'avant correction sont archivés dans `empreintes/pre_correction/`. Verdict : comptages de violations inchangés partout, taux de repli et livraison déplacés d'au plus 0,04 point, mur de compilation déplacé d'exactement un point de grille du pilote — comme la correction le prédisait — et non-régressions revérifiées sur les données nouvelles. Toutes les conclusions qualitatives tiennent ; les chiffres de cette version sont ceux du rejeu. Protocole complet et valeurs avant/après : CHANGELOG du dépôt.

Deux *conceptions* du témoin CRN ont en outre échoué honnêtement en amont des campagnes (section 4.2) — itérations de conception détectées avant tout usage, non comptées parmi les sept défauts. Le motif est constant : les tests unitaires valident des composants contre leur spécification ; ces défauts vivaient dans la composition — politique de tampon $\times$ politique de gel, contrôle de

compilation $\times$ délais d'armement réels, plomberie du filtre $\times$ bras expérimentaux — et seuls la revue adverse et un protocole de falsification pré-enregistré les ont attrapés. Nous y lisons un argument pour la méthodologie, pas seulement une anecdote.

7. Portée et limites

Le bras de contrôle est à son plafond. Dans un régime où l'enveloppe seule est déjà parfaite, un mécanisme de durcissement ne peut que coûter — l'expérience n'a pas de marge de manœuvre dans laquelle H4 aurait pu montrer un bénéfice. La falsification vaut donc exactement pour le régime qu'elle couvre : elle ne démontre pas que le durcissement par l'incertitude est inutile là où l'enveloppe *ne* suffit *pas* (erreur de modèle plus profonde, dynamique réellement adverse, horizons plus longs), seulement que le déployer là où l'enveloppe suffit n'achète rien.

Pourquoi l'enveloppe suffit ici — et où elle ne suffirait pas. La raison est structurelle, pas circonstancielle. La politique de repli — extinction du payload — supprime le terme de décharge dominant, donc son autorité sur l'état contraint est grande devant la perturbation et l'erreur d'estimation, et la marge de garde lui achète le temps d'agir avant tout franchissement de seuil brut. Dans ces conditions,

aucune information d'incertitude ne peut améliorer une enveloppe qui n'est jamais en retard : la plante tire une capacité jusqu'à moitié de la valeur nominale, et l'enveloppe attrape encore chaque cas. L'énoncé généralisable est donc un critère de conception plutôt qu'un rapport de scénario : *le durcissement par l'incertitude de premier ordre ne peut payer que là où l'autorité du repli est marginale relativement au temps-avant-violation*. Cette condition — pas la difficulté du scénario en abstrait — définit le régime dans lequel H4 mérite un test équitable, et elle est testable avant toute campagne : elle dépend du délai d'armement, de la marge de garde et de l'amplitude de la perturbation, pas du résultat.

La condition est mesurée, pas seulement énoncée. Le $\tau_{\text{violation}}$ formel du scénario vaut 205,8 s sur le modèle nominal ; le ratio $r = \tau_{\text{armement}} / \tau_{\text{violation}}$ vaut donc 0,58 en P2. La campagne P3.1 (section 5.4) l'a poussé à 0,68, 0,83 puis 0,92 — l'enveloppe seule reste à zéro violation (0/300 à chaque point) — et au-delà de $r \approx 0,95$ ($\tau_{\text{armement}} = 196$ s), la validation à la compilation refuse le jeu : le régime où l'autorité du repli devient marginale n'est pas un régime difficile à survivre, c'est un régime que RA-FUN-005 interdit de déployer. Tester H4 équitablement exigera donc une autre politique de repli — une politique qui ne supprime pas le terme de perturbation dominant —, pas un autre réglage du délai d'armement.

Puissance statistique. Zéro violation observée sur N runs est une borne supérieure, pas une preuve ; P2.3 resserre la borne de Wilson de 10,7 % (N = 32) à 1,3 % (N = 300). Les intervalles de confiance sur le taux de violation par cycle partagent la même limite. Les distributions de taux par run sont gonflées en zéro et asymétriques à droite, de sorte que les intervalles de Student rapportés dans les tables 3 et 4 doivent se lire comme indicatifs pour le bras A ; les comparaisons entre bras qui portent la conclusion n'en dépendent pas.

Les seuils opérationnels ne sont pas arbitrés (section 4.5) : la falsification n'en a pas besoin, mais la question opérationnelle — où placer le plafond de repli et le plancher de livraison — en a besoin, et elle appartient à un opérateur.

Dettes du prototype. P0 suppose un modèle de prédiction parfait au sein de la famille paramétrique de la campagne (l'écart modèle-monde entre comme dispersion paramétrique et bruit, pas comme erreur structurelle), une action scalaire sous hypothèse de monotonie, un σ constant sur l'horizon, et un niveau de confiance p_S déclaré mais non calibré. L'atteignabilité du repli est vérifiée par échantillonnage aux frontières, pas par atteignabilité formelle. Les bornes de mémoire et de WCET sont préparées (dichotomie à itérations fixes, tampons à capacité fixe) mais ne seront démontrables qu'après le portage C des blocs B2–B4.

8. Conclusion

Un moniteur d'assurance runtime pour satellites autonomes a été spécifié, implémenté comme prototype de référence avec traçabilité au niveau exigence, et soumis à la discipline que sa propre spécification exigeait : un critère de falsification figé avant la campagne, des tirages de plante défavorables, des nombres aléatoires communs avec témoin vérifié, une estimation d'incertitude non circulaire, et des configurations versionnées qui conservent chaque campagne. Le pari que le durcissement par l'incertitude de premier ordre paierait son coût est falsifié dans le régime où l'enveloppe de sécurité suffit — le moniteur à enveloppe seule ne viole rien, et tout niveau d'activation du durcissement coûte plus de repli et moins de livraison mission. La campagne P3.1 resserre encore la portée : pousser le ratio r de 0,58 à 0,92 ne réveille pas H4, et le régime où il aurait sa chance est refusé à la compilation ($r \approx 0,95$) — non déployable par construction.

La formulation honnête est aussi celle qui est publiable : la revendication est bornée à son régime, la décomposition entre coût du pessimisme et coût du seuil est explicite, et les seuils qui transformeraient les courbes en décision d'acceptation opérationnelle sont laissés, délibérément, à un opérateur. Les continuations naturelles sont la connexion calibrée entre k_σ et la distribution d'erreur réelle de l'estimateur, le programme quadratique à action vectorielle du filtre de sécurité prédictif complet, et l'atteignabilité formelle du repli. La continuation qui compte le plus pour H4 elle-même est un régime satisfaisant la condition de la section 7 — autorité du repli marginale relativement au temps-avant-violation. Allonger le délai d'armement n'y suffit pas : la campagne P3.1 a montré que la validation à la compilation refuse le jeu au-delà de $r \approx 0,95$. Il faut donc un repli qui ne supprime pas le terme de perturbation dominant — une autre politique, pas un autre réglage — où la même méthodologie à critère figé pourra donner à l'hypothèse la chance équitable dont elle n'avait pas besoin ici.

Références

- [1] Aurandt, A., Jones, P. H., & Rozier, K. Y. (2022). Runtime verification triggers real-time, autonomous fault recovery on the CySat-I. In J. V. Deshmukh, K. Havelund, & I. Perez (Eds.), *NASA Formal Methods (NFM 2022), Lecture Notes in Computer Science* (Vol. 13260, pp. 816–825). Springer. https://doi.org/10.1007/978-3-031-06773-0_45
- [2] Seto, D., Krogh, B., Sha, L., & Chutinan, A. (1998). The Simplex architecture for safe online control system upgrades. In *Proceedings of the 1998 American Control Conference* (Vol. 6, pp. 3504–3508). IEEE.
- [3] Sha, L. (2001). Using simplicity to control complexity. *IEEE Software*, 18(4), 20–28. <https://doi.org/10.1109/MS.2001.936213>
- [4] Wabersich, K. P., & Zeilinger, M. N. (2018). Linear model predictive safety certification for learning-based control. In

- 2018 *IEEE Conference on Decision and Control (CDC)* (pp. 7130–7135). IEEE.
- [5] Wabersich, K. P., & Zeilinger, M. N. (2021). A predictive safety filter for learning-based control of constrained nonlinear dynamical systems. *Automatica*, 129, 109597. <https://doi.org/10.1016/j.automatica.2021.109597>
- [6] ECSS-E-ST-70-11C Rev.1 (15 octobre 2025). *Space segment operability*. ECSS Secretariat, ESA-ESTEC.
- [7] ECSS-E-ST-70-41C (15 avril 2016). *Telemetry and telecommand packet utilization*. ECSS Secretariat, ESA-ESTEC.
- [8] RAM-SPEC-0001 v0.1 (2026). *Moniteur d'assurance runtime embarqué — spécification fonctionnelle*. Document de travail interne, non publié.
- [9] RAM-NOTE-0001 (2026). *Budget de trace de décision — note de calcul*. Document de travail interne, non publié.
- [10] RAM-ECSS-0001 (2026). *Mapping ECSS du moniteur d'assurance*. Document de travail interne, non publié.
- [11] Bey, S. (2026). *Runtime assurance embarqué sous contrainte spatiale — état de l'art*. Document de travail interne, non publié.